HyperLogLog: How Apps Count a Billion Unique Things in About 1.5 KB

Kabui, Charles

2026-09-15

Read at ToKnow.ai



HyperLogLog answers a question that gets expensive fast: how many different things have I seen? Counting exactly means remembering everything you have already seen, so an analytics service stores millions of visitor IDs just to report how many unique people showed up. HyperLogLog skips that. It hashes each item and looks at the longest run of leading zeros, the way a long run of coin flips hints at how many flips you made. Spread the items across a few

thousand tiny counters, then combine their estimates into one count. The original 2007 paper puts a billion-plus distinct items within about 2% error using 1.5 KB of memory; Redis ships a 12 KB version at 0.81% error.

Counting unique visitors exactly forces you to keep every identifier around, which turns a simple metric into a privacy problem. HyperLogLog keeps only the compact state it builds, so the IDs can be thrown away. Those states also merge, so adding up uniques across a day's worth of servers means combining a few kilobytes per server instead of shipping raw records. The same trick quietly sits inside Redis, BigQuery, ClickHouse, Presto, Druid, and Spark, usually behind a name like approx_count_distinct.

HyperLogLog has barely changed in twenty years. What has changed is that systems now reach for the approximate count first, saving exact counts for the numbers that must be right. Once a small error is acceptable, questions that used to demand a database fit into a few kilobytes.

Read More: Bloom Filters: How Apps Remember a Billion Things in About a Gigabyte

Sources:

- Wikipedia: HyperLogLog
- Redis Docs: HyperLogLog
- antirez: Redis new data structure, the HyperLogLog
- Flajolet et al. (2007): HyperLogLog, the analysis of a near-optimal cardinality estimation algorithm (PDF)
- BigQuery: HyperLogLog++ functions

Disclaimer: *For information only. Accuracy or completeness not guaranteed. Illegal use prohibited. Not professional advice or solicitation.* ***Read more:*** */terms-of-service*